Mini-Project 3
Adversarial Search for Tic-Tac-Toe

By Skynet Group: Nickolas Diaz, Ethan Ashton, Andres Saldarriaga, Caitlyn Sherman, Tris Replogle

Table of Contents

Introduction

In this mini project we will implement two AI agents that play Tic-Tac-Toe and discuss how they work and build them out in Python. Lastly, we will test and measure how well they do against each other and how much computation/time they use.

Part 0: We will explain how Tic-Tac-Toe and build a class that allows our agents to play and validate the rules, turns and winners/losers.

Part 1: We will implement two agent approaches first is a minimax algorithm with alpha-beta pruning.

Part 2: Implement a Monte Carlo Tree Search agent, which is a simulation-based algorithm that estimates the quality of each move by repeatedly playing random games to completion.

Part 3: We will have a face off against the two agents plus a perfect agent and a random agent. Next, we will analyze the amount of time each agent performs.

Part 0

Game engine, correct and tested

Tic-Tac-Toe is a simple game 2 player game where the board is a 3x3 grid of squares, where each player can place an empty spot on the grid either with a “X” or “O”. To win a player must have 3 straight marks, either horizontally, vertically, or diagonally. If all 9 spaces are filled, it is a tie.

Example game:

*	*	*
X	*	*
*	*	*

*	*	O
X	*	*
*	*	*

*	*	O
X	*	*
*	X	*

*	*	O
X	O	*
*	X	*

*	*	O
X	O	*
X	X	*

*	*	O
X	O	*
X	X	O

X	*	O
X	O	*
X	X	O

X wins as they have 3 in a row

Code implementation for Tic-tac-toe game state:

The board is a list of 9 integers, index representing, since it is zero index and a 1d array

0, 1, 2,
3, 4, 5,
6, 7, 8
The current player is a 1 for X or a -1 for O, and 0 for an empty spot

Functions:

- Get legal moves: returns a list of indexes where the board is empty
- Make move: inputs a move as an index and returns a new Tic-tac-toe state
- Check winner: returns 1 for win, -1 for loss and 0 for otherwise
- Display: prints out the status of the board

Part 1

Basic minimax, correct play

Tic-Tac-Toe, although simple, its potential combinations are numerous, think there are 9 ternary options for the game board, making the calculation of 3 ^ 9 = 19,683 possible board combinations. Talking about potential moves is around 9 factorials, equaling 362,880 possible moves, though the actual number is much smaller since it includes moves that are played after winning. Since there are so many possible combinations of moves using a decision tree would be too hard to implement since there are too many options.

The Mini-Max Algorithm would be a solution to solve Tic-Tac-Toe, it is a decision-making algorithm, used to minimize the possible loss in a worst-case scenario and maximize the potential gain. It involves 2 players/agents, if it is the player's turn, pick the max move where the most utility is gained, if it is the opponent's turn, pick the min move where the least utility is gained.

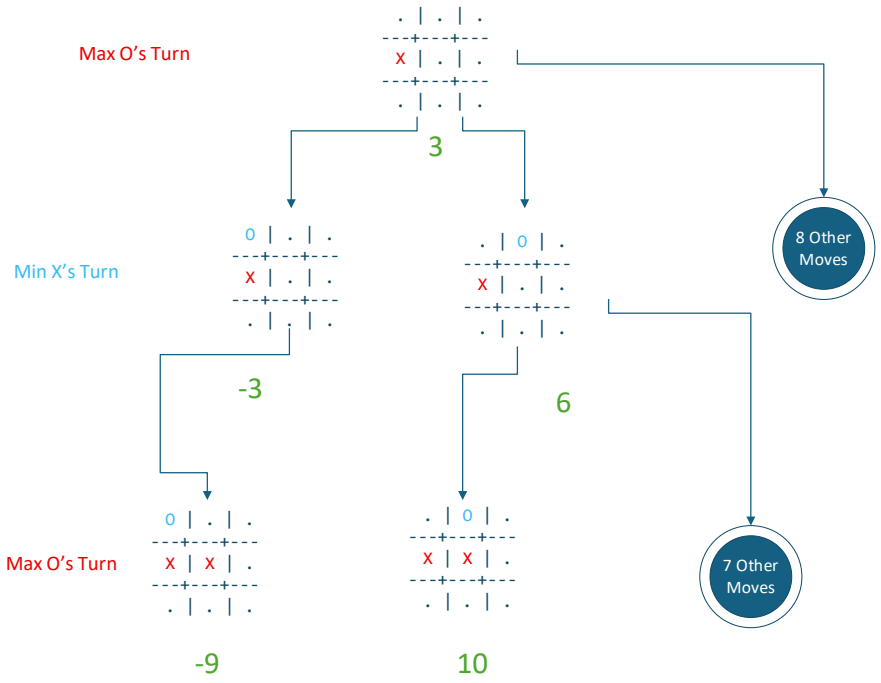
Functions for the Tic-Tac-Toe Mini-Max Algorithm:

- Mini-Max: inputs a Tic-tac-toe state and returns the best move as an integer,

If it is the player's turn it loops through all the possible legal moves, calls the function min_value, and from the list it returns the best move

If it is the opponent's turn it loops through all the possible legal moves, calls the function max_value, from the list it returns the best move

- max_value: inputs a Tic-tac-toe state and returns the best move as an integer, it recursively loops through all the legal moves calling min_value and getting the max of it, the base case is when the Tic-tac-toe game is finished where it returns a number based on whether it lost, tied or won.
- min_value: inputs a Tic-tac-toe state and returns the best move as an integer, it recursively loops through all the legal moves calling max_value and getting the min of it, the base case is when the Tic-tac-toe game is finished where it returns a number based on whether it lost, tied or won.



Verification

From the empty board, minimax for X should return to a corner or the center. TRUE

```
 X | . | .
---+---+---
 . | . | .
---+---+---
 . | . | .

From the empty board, minimax for X should return to a corner or the center. True
```

If X plays center and O plays a corner, X should respond with the opposite corner.

```
 X | . | .
---+---+---
 . | X | .
---+---+---
 . | . | O

If X plays center and O plays a corner, X should respond with the opposite corner. True
```

Two minimax agents playing with each other should always draw.

```
 X | X | O
---+---+---
 O | O | X
---+---+---
 X | O | X

Two minimax agents playing with each other should always draw. True
```

Alpha-beta pruning

Alpha-Beta Pruning: further optimizes the Mini-Max Algorithm, it prunes branches that do not need to be calculated. It uses two extra inputs, one for alpha which is the best value that the maximizing player can guarantee and beta which is the best value the minimizing player can guarantee. If one of the player's options is lower than the alpha then it is skipped, also If one of the opponents' options is higher than the alpha then it is skipped. This pruning greatly improves the computational side of running it since it can get exponentially expensive over time.

Node Counts and Questions

A count of the amount of nodes the Mini-Max vs Mini-Max with pruning is calculated over it running. The results are shocking.

The total number of nodes evaluated by plain minimax: 549,947

The total number of nodes evaluated by alpha-beta: 18,298

The percentage of nodes pruned: 96.67%

Questions

Why does minimax always produce a draw in Tic-Tac-Toe, regardless of who moves first? Tic-Tac-Toe is a solved game, and if both players are optimal then it will always result in a tie. Since Minimax has seen every game, it will always play optimally thus resulting in a tie between two minimax agents.

In alpha-beta pruning, does the order in which moves are examined affect the number of nodes pruned? Why or why not?

Yes, if the move that changes the alpha or beta is changed first, then it will prune more branches. For example, if the best branch was examined the rest of the branches can be pruned.

What is the time complexity of minimax? What is the best-case complexity of alphabet a? Relate your node counts to these theoretical values.

The time complexity of minimax is O(b ^ m) where b is the average of legal moves, and the m is the max depth of the game. The alphabet a is O(b ^ m/2).

Plain minimax: 549,947

Alpha-beta: 18,297

Would minimax be practical for a game like chess or Go? Explain your reasoning.

Minimax would be way too computationally expensive for it to work. Working on something like 40 for “b” and 40 for “m” would be 1.09 e+32 which is far too expensive for this case. It would be useful for something small like a 5x5 chess game.

Part 2

MCTS

The Monte Carlo Tree Search agent is different from the MinMax agent. It focuses on exploring different paths to find its next move. It works through a tree of possibilities to find the move that has the most connected wins. It uses the formula UCB1(n) = U(n)/N(n) + C * sqrt(ln(N(parent)) / N(n)) to find the move with the most wins.

To start implementation, we create the search with nodes and paths. Each of the nodes has a tracker for the number of visits and the wins. The number of unvisited nodes are recorded and updated as the agent travels. Then, to find the node that will lead to a win, it calculates the UCB for the node.

The agent moves through the tree and selects the current node it believes is the best node. The agent expands the tree by going through a path to see the possible results. To simulate this, it tests legal moves. The back propagation is there to make sure it travels back through the tree. Then the agent loops through these steps.

Questions

Run MCTS with 100, 1,000, and 10,000 iterations. How does the quality of play change? Is there a point of diminishing returns for Tic-Tac-Toe?

MCTS was run with 100, 1000, and 10000 iterations. The quality of play went down as the iterations increased somewhere between 1000 and 10000.

Experiment with different values of C (try 0.5, 1.0, 1.41, and 2.0). How does C affect the balance between exploration and exploitation? Report the win rates against a random player for each value.

There were still improvements to be made with the algorithm, so we then started trying to increase the value of c to cause more exploitation and lowering the value to try more exploration.

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	97	0	3
Random (X) vs Minimax (O)	0	77	23
Minimax (X) vs MCTS (O)	12	0	88
MCTS (X) vs Minimax (O)	0	1	99
Perfect (X) vs Random (O)	98	0	2
Random (X) vs Perfect (O)	0	88	12
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	0	100
Random (X) vs MCTS (O)	5	94	1
MCTS (X) vs Random (O)	100	0	0

,

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	97	0	3
Random (X) vs Minimax (O)	0	79	21
Minimax (X) vs MCTS (O)	9	0	91
MCTS (X) vs Minimax (O)	0	2	98
Perfect (X) vs Random (O)	98	0	2
Random (X) vs Perfect (O)	0	85	15
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	3	97
Random (X) vs MCTS (O)	4	94	2
MCTS (X) vs Random (O)	98	0	2

,

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	98	0	2
Random (X) vs Minimax (O)	0	76	24
Minimax (X) vs MCTS (O)	10	0	90
MCTS (X) vs Minimax (O)	0	2	98
Perfect (X) vs Random (O)	100	0	0
Random (X) vs Perfect (O)	0	90	10
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	2	98
Random (X) vs MCTS (O)	4	90	6
MCTS (X) vs Random (O)	99	0	1

,

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	97	0	3
Random (X) vs Minimax (O)	0	78	22
Minimax (X) vs MCTS (O)	12	0	88
MCTS (X) vs Minimax (O)	0	6	94
Perfect (X) vs Random (O)	93	0	7
Random (X) vs Perfect (O)	0	84	16
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	4	96
Random (X) vs MCTS (O)	3	90	7
MCTS (X) vs Random (O)	99	0	1

Why does the algorithm return the move with the most visits rather than the highest win rate? Give a concrete example where the two criteria would select different moves.

This algorithm chooses the most visited spot because the spot with the most visits has more options for winning. This is different from the win rate as it could take an option

different from one that would allow it to directly win. For example, it could block a move instead of making a final winning move.

MCTS does not require an evaluation function; it only needs the rules of the game. Why is this an advantage for certain games?

Not needing an evaluation function means that it will spend less time double checking the move it has made and that it will explore options that other algorithms would not choose.

Part 3

Tournament, Timing, and Analysis

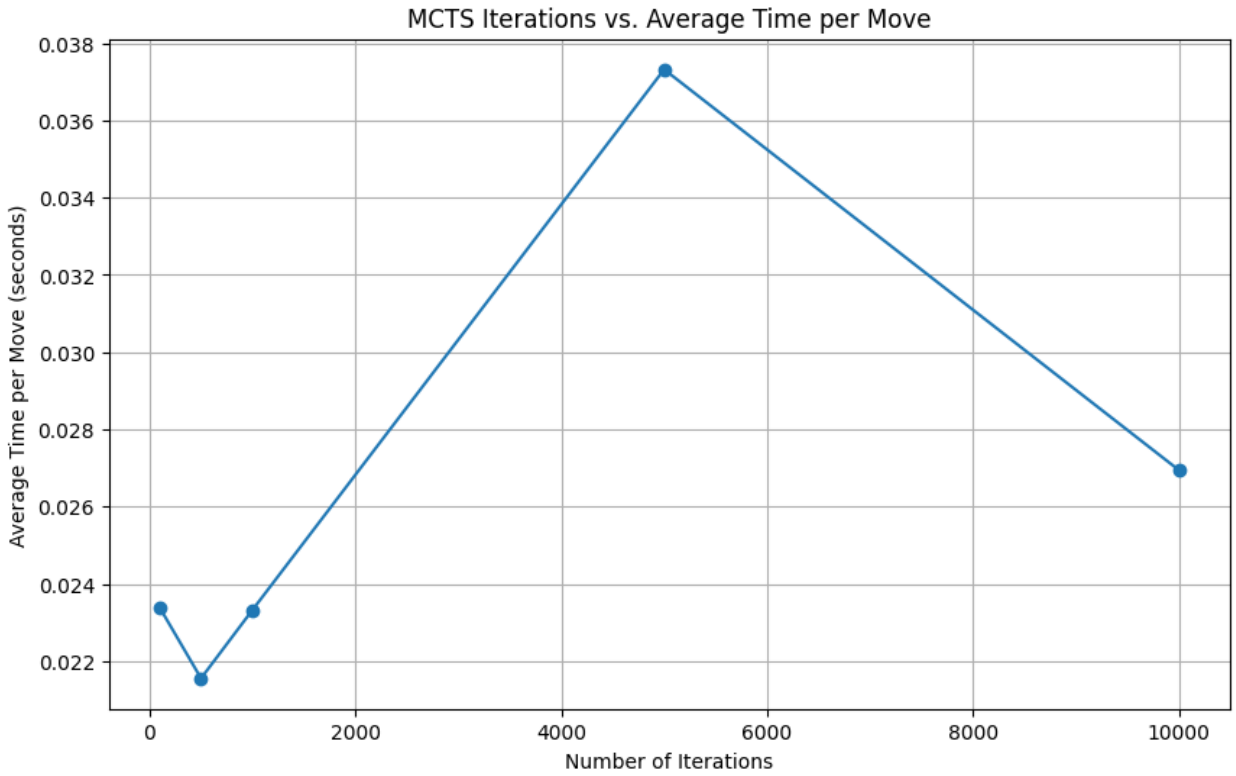
To test the different agents, a tournament style game was created. In this tournament, the agents would play against each other as X or O. The agents played 100 games for each instance. For example, MinMax (playing as X) would go against Random (playing as O) then after they play 100 games this way they would switch with MinMax playing O and Random playing X. We would then record the number of games that ended in wins(o), wins(x), and draws for each agent pair. After that we wanted to compare the time for each move calculation in the MCTS vs in the MinMax. We took the time for iterations from 500 to 10000 for MCTS and compared them to MinMax. MinMax started off with slower times but quickly outpaced the MCTS agent after 2000 iterations.

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Random (O)	992	0	8
Random (X) vs Minimax (O)	0	808	192
Minimax (X) vs MCTS (O)	111	0	889
MCTS (X) vs Minimax (O)	0	47	953
Random (X) vs MCTS (O)	43	914	43
MCTS (X) vs Random (O)	989	0	11

3.2

For the 3 agents, we got varying results on average time. The minmax agent took on average 0.0054329 seconds. The random model was the fastest on average taking a total of 1.8775463e-06 seconds, then the slowest was the MCTS model taking on average 0.0158519 seconds.

For modeling the time per move for MCTS based on the number of iterations it took, we came out to this graph.



Questions

Which agent wins more games against the random player? Are the results what you expected?

The agent that wins against random the most would be the MCTS agent for playing X and Y. Which was expected.

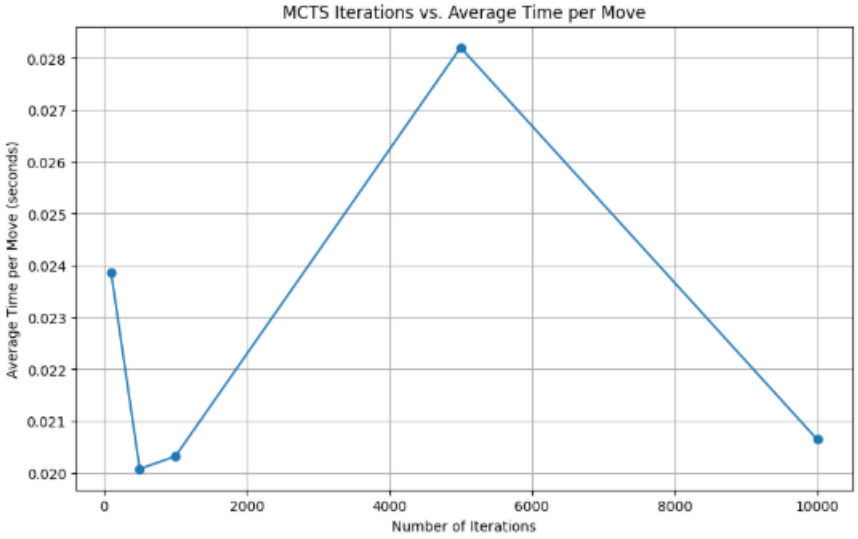
When minimax plays against MCTS, what happens? Does MCTS ever lose? If so, how many iterations does it need to avoid losing?

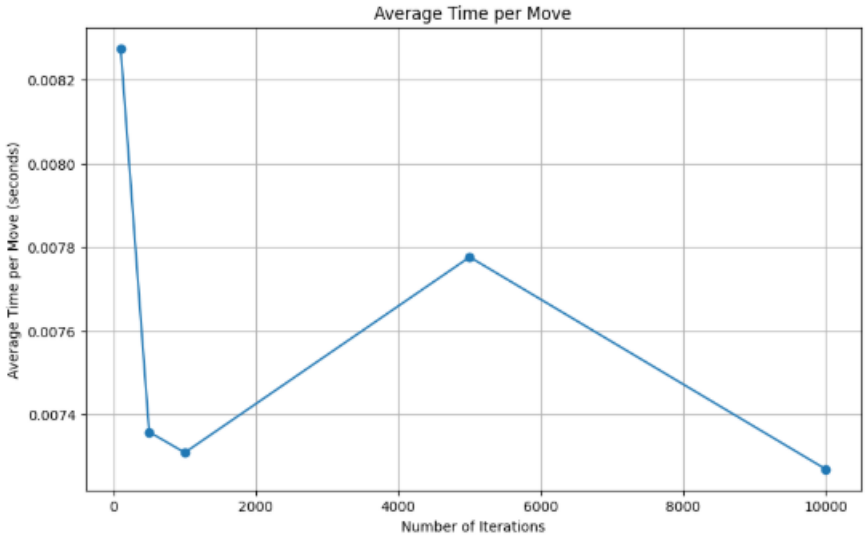
MCTS lost 10 times when O and 2times when X against MinMax and ties for the rest of the 100 times. We could not find an interation amount that led to a win for MCTS.

Matchup	X Wins	O Wins	Draws
Minimax (X) vs Perfect (O)	0	0	100
Perfect (X) vs Minimax (O)	0	0	100
Minimax (X) vs Random (O)	98	0	2
Random (X) vs Minimax (O)	0	76	24
Minimax (X) vs MCTS (O)	10	0	90
MCTS (X) vs Minimax (O)	0	2	98
Perfect (X) vs Random (O)	100	0	0
Random (X) vs Perfect (O)	0	90	10
Perfect (X) vs MCTS (O)	100	0	0
MCTS (X) vs Perfect (O)	0	2	98
Random (X) vs MCTS (O)	4	90	6
MCTS (X) vs Random (O)	99	0	1

Compare the time per move for both agents. At what iteration count does MCTS become slower than minimax for Tic-Tac-Toe?

After 2000 iterations, MCTS begins to become slower than the minmax agent.





Consider a game with a much larger branching factor, such as Connect Four (branching factor of approximately 7, depth up to 42). Which algorithm would you choose for that game, and why?

In games with large branching factor, you would want to use MCTS as its focus on promising further results on trees instead of fully exploring every single option means that it will more efficiently be able to find optimal solutions, especially compared to minmax.

Under what circumstances would you prefer MCTS over minimax? Under what circumstances would minimax be the better choice?

MCTS works better in scenarios where evaluation functions may be too complex to create, when the problem features a large branching factor, or when there's limited time or computational power to use with the model. Minmax tends to be better in scenarios with low branching factors, ones with simple and strong evaluation functions, and ones where high precision is required in moves.